THE ONE-LINER

An event-driven ingestion pipeline: an S3 upload fires a Lambda that routes each file by type to a streaming handler — which hashes it, stores it, and records it — with a **reconciler** to mop up partial failures.

THE SPINE — WHAT YOU DRAW

1. **Entry box — the handler signature and what fires it.**
   `processUpload(key, bucket)` — the Lambda / API handler, triggered by the S3 **ObjectCreated** event.
2. **Routing — how a file type picks its handler.**
   `extname(key)` → the `strategies` map (jpg, mp4, ttf, bin, zip…). An **O(1) lookup**, never a switch.
3. **The branch for an unrecognized type.**
   unknown ext → `skip` (a logged no-op); a match → the format handler.
4. **Inside a handler — the single-read data flow.**
   `readStream → PassThrough → [hash | s3.upload] → Promise.all`. **One disk read**, forked two ways.
5. **The export path, and its memory property.**
   `cursor(batch 100) → csv → res`, the backpressure loop drawn back from socket to cursor — **constant memory** at any row count.
6. **The import path — the id-collision algorithm.**
   The 4-tier ladder — **REUSE / REMAP / REGEN / INSERT** — plus the `oldId→newId` FK remap applied before each child insert.
7. **The dual-write caveat, and its fix.**
   Two stores, no shared txn → track the S3 keys, **compensating-delete** on rollback. (The one people forget.)
8. **The backstop for orphans — and its one guard.**
   A **reconciler** sweeps S3 for keys with no DB row → delete — but only past a grace window / **PENDING** marker, so it never touches an in-flight upload.
9. **How a redelivered event avoids double work.**
   At-least-once delivery → a **processed-marker** (conditional put on the content hash). A replay sees it and no-ops — the effect is idempotent.

DECISIONS & SWITCH CONDITIONS

Lambda / object → **SQS + workers** when you need **retries, a DLQ, ordering**, burst-smoothing, or jobs over 15 min.

During (fork) → **After / two-pass** when the file is tiny, or an API needs the **digest before** you can store it.

SQLite → **JSON** when data is small, human-readable, with **no** binary or relations.

Single PUT → **Multipart** when large objects, flaky networks, or you want **per-part retry** and resumability.

Compensating delete → **Reconciler** when you need a **durable backstop** that eventually catches every orphan.

Sync inline → **Async queue** when heavy or bulk work where **eventual** completion is fine.

Reconciler (heal) → **Outbox (prevent)** when correctness must be **airtight** and you can anchor on one DB transaction — commit the row + an event atomically, and a relay does the S3 work with retries off **one commit point**.

CEILINGS — THE NUMBERS

Average throughput: 116 /s — objects/day ÷ 86,400 seconds

Peak throughput: 1,157 /s — average × 10 peak ratio

Lambda concurrency at peak: 2,315 — exceeds the 1,000 default — RDS Proxy, or buffer through SQS

DB connections at peak: 2,315 — far past a Postgres pool (~100) — needs RDS Proxy or a queue

Storage written / day: 20 TB — 7.3 PB per year of raw objects

S3 PUTs / day: 10,000,000 — ≈ $50.00/day in PUT requests alone

The number you say isn't the point — the **ceiling** it reveals is. Concurrency past 1,000 says 'buffer through SQS'; connections past the pool say 'RDS Proxy.'

TRAPS → THE FIX

"I'd query all the rows, then write them to the CSV."	→	Server-side cursor, 100 rows at a time, piped with backpressure — **constant memory** at any row count.
"I'll store the file, then read it back to hash it."	→	A **PassThrough fork** — one read feeds the hash and the upload at once.
"A switch statement routes each file type to its handler."	→	An **O(1) dispatch map**. A new type is one entry and zero router changes.
"It writes to S3 and Postgres, so the data's consistent."	→	Track created keys, **compensating delete** on failure, and a **reconciler** as the durable backstop.
"Exactly-once delivery means I won't double-process."	→	Idempotent **effects** — a content-hash key or a processed-marker check-and-set. Replays no-op.
"On any failure, I just retry."	→	Idempotency + a **DLQ** + backoff + a per-job timeout, so one bad input fails fast instead of hanging.
"Lambda per object — that's the design."	→	Name the **switch condition** — move to SQS the moment you need retries, a DLQ, or ordering.
"A reconciler deletes any S3 key with no DB row."	→	A grace window, or a **PENDING marker**, so the reconciler never touches in-flight work.
"...so I'd use a Lambda and an S3 trigger and—"	→	**Frame first** (scope + load), then design, then name the **resource that gives out first**. Reasoning visible beats architecture recited.

SENIOR TELLS — SAY THESE

- Name the **switch condition**, don't defend one side.
- The **PassThrough fork** is the only path that survives a 500 MB object.
- JSON base64-bloats binary **~33%** and loses referential integrity.
- Multipart resumes **per part** — pair it with a lifecycle rule to abort orphans.
- Run delete **and** reconciler; reach for 2PC only when atomicity is non-negotiable.
- Two-tier by interactivity: **one dispatch map, two execution venues**.
- The outbox needs a DB transaction to anchor on; the reconciler is the honest backstop for when the stores genuinely can't share one.

HARDER ANGLES — CURVEBALL-READY

Exactly-once — Reframe the premise out loud, then give the real mechanism.

Security — Name the defenses in layers, then the one that contains an input that slips through.

Cost — Profile before optimizing, then name the single biggest lever.

Backfill — Reprocess at scale without melting live traffic — and say why retry is safe.

Ordering — Reconcile unordered delivery with an ordering need — and question whether you need it.

Multi-region — Name what goes cross-region, the consistency you can actually promise, and the honest RPO.

Silent failure — Trace one item's journey and name where you'd look, in order — plus the silent-failure suspects.

Deploy under load — Name what makes the rollout safe, and the one property the pipeline already needs for it to work.

IF THEY SAY "QUICKLY" — THE 30 SECONDS

Operators push content to S3, which triggers the processor. It routes on file type through an **O(1) strategy map** — not a switch — so a new format is a one-line add. Each handler **streams the file once** and forks that read to hash and upload in parallel, so memory stays flat no matter the size. The catch: the object store and the database share no transaction, so I track written keys and **compensate on failure**, with a reconciler as the backstop. And since every trigger is **at-least-once**, processing is idempotent — a replay no-ops on a content-hash marker.